Web Application Documentation

Project Team

September 15, 2025

1 Introduction

This document provides comprehensive documentation for the Web Application project, a full-stack web application built using modern technologies and best practices.

2 Project Overview

The application is a web-based system that provides user authentication and authorization capabilities. It demonstrates the implementation of secure user management with features like signup, login, and protected routes.

3 Architecture

The project follows a microservices architecture with three main components:

3.1 Frontend Service

- Built with React 19 and TypeScript
- Uses React Router for client-side routing
- Implements protected routes for authenticated users
- Communicates with backend via RESTful API using Axios
- Features responsive design for various screen sizes

3.2 Backend Service

- Built with NestJS framework
- Implements RESTful API endpoints
- Uses JWT for authentication

- Includes Swagger/OpenAPI documentation
- Follows modular architecture with separate modules for auth and user management

3.3 Database Service

- MongoDB for data persistence
- Stores user data and authentication information
- Mongoose ODM for database interactions
- Supports data validation and schema enforcement

4 Technologies Used

4.1 Frontend

- React
- TypeScript
- React Router
- Axios
- Create React App

4.2 Backend

- NestJS
- TypeScript
- Passport JWT
- MongoDB with Mongoose
- Swagger/OpenAPI

4.3 Database

- MongoDB 4.4+
- Mongoose ODM
- MongoDB Atlas (optional for cloud deployment)

4.4 DevOps

- Docker
- Docker Compose
- Nginx (for frontend serving)
- Node.js 18+

5 API Documentation

5.1 Authentication Endpoints

- POST /auth/signup
  - Creates new user account
  - Required fields: username, email, password
- POST /auth/login
  - Authenticates user and returns JWT token
  - Required fields: email, password

5.2 User Endpoints

- GET /user/profile
  - Returns authenticated user's profile
  - Requires valid JWT token
- PUT /user/profile
  - Updates user profile information
  - Requires valid JWT token

6 Security

The application implements several security measures:

- JWT-based authentication
- Password hashing using bcrypt
- Protected routes in both frontend and backend
- CORS protection
- Environment-based configuration

7 Deployment

The application can be deployed using either Docker Compose or traditional npm-based deployment. Docker Compose is recommended for production environments as it ensures consistency across different deployment environments.

7.1 Docker Compose Deployment

Includes:

- Frontend container (Node.js 18+ for build, Nginx for serving)
- Backend container (Node.js 18+)
- MongoDB container
- Docker network for service communication
- Volume for MongoDB data persistence

7.2 Traditional Deployment

Requirements:

- Node.js 18 or later
- npm 8 or later
- MongoDB 4.4 or later